

Social Mini App — Nimiq Mini Apps Competition (Cycle 2)

Tip-native social app built on the Nimiq Pay Mini Apps Framework. 2-builder team:

Builder A = Frontend · Builder B = Backend

Prize target: max out the scorecard — **45 Functionality/Reliability/Usefulness · 25 Nimiq Pay Integration · 15 Real Usage · 10 Design & UX · 5 Promotion**

1. Feature Scope (locked)

Core (must be flawless):

- Wallet connect (Nimiq Pay)
- Editable profile (avatar, username, bio)
- Light/dark mode
- Create post
- Feed (view posts)
- Like
- Comment
- Tip a post/user (real Nimiq Pay transaction — this is the scoring centerpiece)

Retention layer (locked additions):

- Leaderboard — daily/weekly top tippers & top earners
- Streaks & badges — posting/tipping consistency
- Notifications — in-app ("X tipped your post", "X commented", etc.)

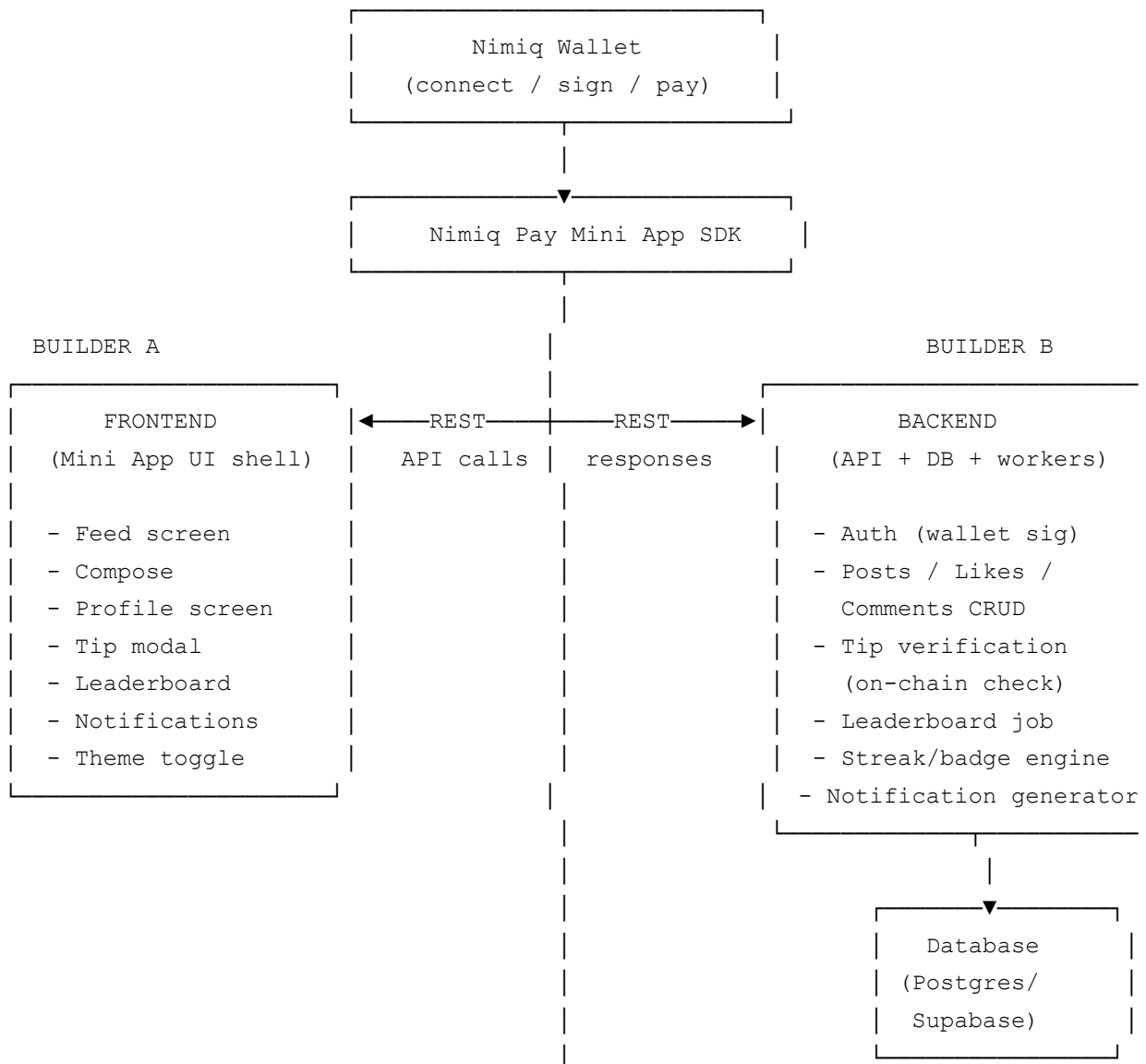
Stretch goals (only after core is bulletproof):

- Repost
- Bookmark

- Tip-to-boost (tipping pushes a post up a “trending” rail)
- Pinned “top supporter” comment on a post

Cut ruthlessly if time runs short — a smaller app that never breaks beats a big one that half-works. Judges score reliability higher than feature count.

2. System Schematic



The one rule that keeps you both unblocked: neither of you touches the other’s code. You talk only through the **API contract** (Section 4). As long as the contract doesn’t change, Builder A can build against fake/mock data and Builder B can build against Postman/curl — and everything plugs together on sync day.

3. Tech Stack (proposed — confirm together in Phase 0)

Layer	Choice	Why
Frontend	Next.js (or plain React/Vite)	Fast, works well inside Mini App wrapper
Wallet/Payments	Nimiq Pay Mini Apps SDK	Required by competition
Backend	Node.js (Express/Fastify)	Fast to ship, matches JS frontend
Database	Postgres (Supabase)	Free tier, instant hosted DB + auth helpers
Hosting (backend)	Render / Railway free tier	No card required, easy deploys
Hosting (frontend)	Vercel	Free, auto-deploys from GitHub
Auth	Wallet signature challenge (sign message → verify → session token)	No passwords, native to Web3

4. API Contract (defined once in Phase 0 — this is what unblocks parallel work)

Builder B implements these. Builder A builds the entire UI against these shapes using mock data until the real backend is ready.

```
POST    /auth/challenge    → { message } (nonce to sign)
POST    /auth/verify      → { token, user }

GET     /profile/:wallet  → { username, bio, avatarUrl, badges[], streak }
PUT     /profile          → update own profile

GET     /feed?cursor=     → { posts[], nextCursor }
POST    /posts            → create post
GET     /posts/:id        → single post + comments

POST    /posts/:id/like   → toggle like
```

```

POST    /posts/:id/comment      → { text }
POST    /posts/:id/repost   → (stretch)
POST    /posts/:id/bookmark → (stretch)

POST    /tips               → { toWallet, postId?, amount, txHash }
GET     /tips/leaderboard?range=daily|weekly

GET     /notifications      → list
POST    /notifications/read → mark read

GET     /streaks/:wallet    → current streak + badges

```

Exact field names can flex — just **agree and freeze them together before Phase 1 starts**, then log any change in this file so both sides stay in sync.

5. Build Phases

Each phase has an **A track** and a **B track** that run at the same time without touching each other, followed by a **Sync Point** — a short joint test before moving on. We go one step at a time; I'll check with you before we move past each step.

Phase 0 — Planning & Setup (*joint*)

- Confirm final feature list (Section 1)
- Confirm tech stack (Section 3)
- Freeze the API contract (Section 4)
- Read Nimiq Mini Apps docs together (nimiq.dev/mini-apps)
- Set up GitHub repo (recommend: monorepo with `/frontend` and `/backend` folders)
- Each builder gets a Nimiq wallet + joins the Skool community
- Agree on env var names / secrets handling

Phase 1 — Foundation (*parallel*)

Builder A (you):

- App shell + routing (Feed / Profile / Leaderboard / Notifications screens, empty states)
- Light/dark theme toggle
- Wallet connect UI (against mocked `/auth` responses)

Builder B:

- DB schema: users, posts, likes, comments, tips, notifications, streaks
- API server scaffold + hosting deploy skeleton
- Real wallet-signature auth (`/auth/challenge` , `/auth/verify`)

Sync Point 1: Connect Builder A's wallet UI to Builder B's real auth endpoints. Confirm login works end-to-end.

Phase 2 — Profile + Posting Core Loop (*parallel*)**Builder A:**

- Profile edit screen (avatar/username/bio)
- Compose-post UI
- Feed UI rendering real post objects

Builder B:

- Profile CRUD endpoints
- Create/read post endpoints
- Feed pagination

Sync Point 2: Create a post and edit a profile end-to-end, confirm it persists and shows in the feed.

Phase 3 — Engagement + Tipping (*parallel*)**Builder A:**

- Like / comment UI
- Tip modal (amount input → triggers Nimi Pay transaction)
- (Stretch) Repost / bookmark UI

Builder B:

- Like/comment endpoints
- Tip endpoint: verify the on-chain transaction, credit sender/receiver, log for leaderboard
- (Stretch) Repost/bookmark endpoints

Sync Point 3: Send a real tip end-to-end (testnet) and confirm it's verified and recorded.

This is the highest-value sync — it's the 25-pt integration score.

Phase 4 — Retention Features (*parallel*)

Builder A:

- Leaderboard screen (daily/weekly toggle)
- Streak + badge display on profile
- Notifications screen + unread indicator

Builder B:

- Leaderboard computation (scheduled job or on-read query)
- Streak tracking + badge award logic
- Notification generation (on like/comment/tip events)

Sync Point 4: Full walkthrough — post, get liked/tipped/commented, see it reflected in notifications, streak, and leaderboard.

Phase 5 — Polish, QA & Submission (*joint*)

- Bug bash on both sides
- Mobile/responsive check (Mini Apps run inside a phone-width wrapper)
- Edge cases: empty states, failed transactions, expired sessions
- Record demo video
- Write submission README (what it does, how Nimi Pay is used, setup instructions)
- Promotion posts in Skool + X for the 5 promotion points

6. Working Process

- We go phase by phase, step by step — no skipping ahead.
- After each step is marked done, I'll ask before we move to the next one.
- When we reach UI work, send designs/screenshots and we'll build against those directly.
- Any change to the API contract (Section 4) gets logged here so Builder B always knows what Builder A expects, and vice versa.

